Razonamiento Automático 2026/27 Challenge 2: Enunciado de entrenamiento

Programar en ensamblador de Game Boy

Objetivos de aprendizaje

De repaso (ya entrenados en el Challenge 1, aquí en un contexto más complejo: subrutinas, corrección de errores ajenos, control de VRAM más fino):

- Ensamblar y ejecutar. Cargar cartuchos.
- Instrucciones básicas y registros
- Condicionales (if)
- Bucles básicos
- Manejar el Tilemap básico
- Detectar el inicio de VBLANK (usando rLY=144)

Nuevos en este Challenge:

- Funciones básicas (`call` , `ret`)
- Usos indirectos de registros (`hl` , `de` , `bc`)
- Aprender instrucciones y operaciones (`bit` , `add` , `sub` , `adc`)
- Leer, reconocer y corregir errores en código
- Diagramas de instrucción: qué se lee y qué se escribe (en las dos direcciones)
- Leer el PAD (hardware básico)

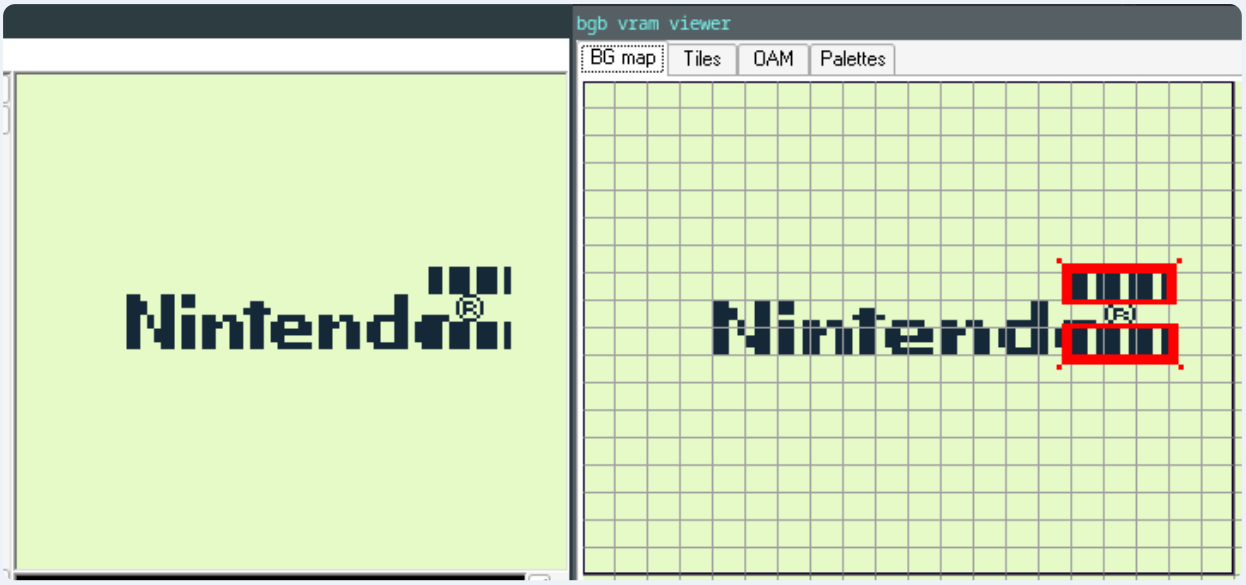
Ejercicio 1

Haz un programa que dibuje los "códigos de barras de Nintendo", como en la figura 1

No puedes usar `ld [nnnn], a`

Son 2 bandas que repiten 3 veces el tile `$0F` **(** `15` **) arriba y abajo de ®.**

Figura 1



Ejercicio 2

Corrige el programa

El programa quiere borrar el logo de Nintendo, pero falla. Lista los errores encontrados, e indica cómo corregirlos.

Asume que `wait_vblank` **está definida aparte**

Row filler

```
def LOGO_LINE_1 equ $9900
def LOGO_LINE_2 equ $9920
def LINE_SIZE   equ 20
def BLANK_TILE  equ 0
;;---------------------------------------
;; Fills a Row with Tiles
;; 📥 Input Values
;;  HL: Start of the row
;;   A: Tile to be written
;;   B: Number of tiles to write
;; ❌ Modifies: HL, B
fill_row:
      ld hl+, [a] ;; Write BLANK at [hl]. HL+=1
      sub b, 1    ;; --B (One tile less)
   jr  z, fill_row
   ret            ;; Return

;;---------------------------------------
;; Clear Nintendo Logo
ej2::
   jr    wait_vblank      ;; Synchronize with VBLANK

   ;; Clear Upper Row
   ld  a, [BLANK_TILE]    ;; A  = Blank tile Number
   ld [hl], LOGO_LINE_1   ;; HL = VRAM Start of Upper Line
   ld LINE_SIZE, b        ;; B = 20 tiles to be cleared
   jp    fill_row
   ;; Clear Lower Row
   ld  [LOGO_LINE_2], hl  ;; HL = VRAM Start of Upper Line
   call fill_row

   di            ;; Stop program
   halt
```

Ejercicio 3

? 3.1. Indica qué se lee y qué escribe al ejecutar estas instrucciones

Ignora los flags H, N.

ⓘ Ejemplo

INSTR.	LEE	ESCRIBE
`ld a, b`	Reg(`B`)	Reg(`A`)
`adc 5`	IMM(`5`), Reg(`F`):Flag(`C`)	Reg(`A`), Reg(`F`):Flags(`C` , `Z`)

- `ld e, $30`
- `cp %00000010`
- `ld [hl], b`
- `jp $0150`
- `sub b`
- `jr z, $08`
- `add [hl]`
- `ld a, [bc]`

3.2. *Escribe la instrucción*

Cada fila describe qué lee, qué escribe y qué hace una instrucción. Escribe la instrucción de ensamblador que le corresponde.

Podría haber más de una instrucción válida: si la tuya no coincide con la solución, comprueba que hace exactamente lo mismo antes de darla por buena.

Ejemplo

LEE	ESCRIBE	QUÉ HACE	INSTRUCCIÓN
Reg(`H`)	Reg(`A`)	Copia el valor de `H` en `A`	`ld a, h`

LEE	ESCRIBE	QUÉ HACE	INSTRUCCIÓN
IMM(`$22`)	Reg(`C`)	Carga el valor inmediato `$22` en `C`	
Reg(`A`), IMM(`%00000101`)	Reg(`F`):Flags(`Z` , `C`)	Compara `A` con el valor inmediato `%00000101` (resta sin guardar el resultado)	
Reg(`E`), Reg(`HL`)	MEM(`HL`)	Guarda el valor de `E` en la dirección de memoria que indica `HL`	
IMM(`$0200`)	Reg(`PC`)	Salta de forma incondicional a la dirección `$0200`	
Reg(`A`), Reg(`D`)	Reg(`A`), Reg(`F`):Flags(`Z` , `C`)	Suma el valor de `D` a `A` y guarda el resultado en `A`	
Reg(`F`):Flag(`Z`), Reg(`PC`), IMM(`$0A`)	Reg(`PC`)**?**	Si el flag `Z` **no** está activo, salta de forma relativa `$0A` posiciones hacia delante	
Reg(`A`), Reg(`HL`), MEM(`HL`)	Reg(`A`), Reg(`F`):Flags(`Z` , `C`)	Resta a `A` el byte guardado en la dirección que apunta `HL` , y guarda el resultado en `A`	

LEE	ESCRIBE	QUÉ HACE	INSTRUCCIÓN
Reg(`DE`), MEM(`DE`)	Reg(`A`)	Carga en `A` el byte guardado en la dirección que apunta `DE`	

Notas:

- Los valores ***inmediatos*** (`$15` , `%00001000` , un destino de salto como `$0200` , o un desplazamiento relativo como `$0A`) forman parte de la propia instrucción: van codificados como bytes en ROM justo después del opcode. Por eso se anotan siempre como ***IMM***(`$15`) — nunca como `MEM(...)` , que reservamos para lecturas/escrituras de una dirección de memoria real (ver nota siguiente). Por ejemplo, `ld a, $15` se codifica como `3E 15` , incluyendo el `$15` como 2º byte de la instrucción.
- Un registro usado para ***direccionar*** memoria (`[hl]` , `[bc]` , `[de]`) también cuenta como ***leído***, además de lo que haya en `MEM(...)` : la CPU necesita su valor para saber a qué dirección acceder. Por ejemplo, `ld [hl], b` lee `Reg(HL)` y `Reg(B)` , y escribe `MEM(HL)` .
- Los saltos ***relativos*** (`jr`) leen además el propio `Reg(PC)` , porque necesitan sumarle el desplazamiento inmediato para calcular el destino. Los saltos ***absolutos*** (`jp`) no: sustituyen `PC` directamente por la dirección inmediata, sin necesitar su valor anterior.
- La indicación Reg(`PC`)**?** (con ***interrogante***) indica que ese registro, el `PC` , puede que sea escrito o puede que no. Las instrucciones condicionales modifican sólo si se cumple la condición. Por eso lo indicamos así.
- Recuerda que `%00010000` , `$10` o `16` son exactamente lo mismo. En todos los casos es el valor `16` . Los valores no cambian por escribirlos en distintos sistemas de numeración. Puedes elegir siempre el sistema de numeración que más te convenga en cada momento.

Ejercicio 4

Reescribe este código
Tu versión debe producir el mismo resultado, cumpliendo las condiciones.

No puedes usar el registro A

Código a reescribir

```
wait_till_A_is_pressed:
   ld  a, $10                 ;;
   ld [$FF00], a              ;; Select Buttons Group 1 (Select, Start, B, A)

   ld a, [$FF00]              ;;
   ld a, [$FF00]              ;; Read buttons, stabilizing input
   ld a, [$FF00]              ;;

   bit 0, a                   ;; Check Bit 0 of A (button A)
jr nz, wait_till_A_is_pressed;; repeat while A is not pressed
;; A is pressed, act
```

Ejercicio 5

Escribe un programa
que dibuje el ***logo maligno*** de Nintendo (desordenado). Fíjate en la captura del volcado de memoria: indica cómo debes rellenarla.

Asume que la VRAM, inicialmente, está rellena de 00

Asegúrate que tu código se ejecuta durante VBLANK

No puedes usar `ld [nnnn], a`

Figura 2

```
VRA0:9900  00 00 00 19 18 17 16 15|14 13 12 11 10 0F 0E 0D|
VRA0:9910  00 00 00 00 00 00 00 00|00 00 00 00 00 00 00 00|
VRA0:9920  00 00 00 00 0C 0B 0A 09|08 07 06 05 04 03 02 01|
```



Estado final de la VRAM y lo que se ve en pantalla

Ejercicio 6

Escribe una rutina de colisiones.
Partiendo del código de abajo, comprueba si player y enemy ocupan la misma posición (colisionan).

Si no hay colisión, devuelve `A=0`
Si hay, devuelve `A != 0` (cualquier valor != 0 sirve)

Todas las instrucciones que lean o escriban en WRAM0, deben usar el registro `bc` o `de`.

check_collision placeholder code

```
SECTION "Data", WRAM0
 player_x: DS 1
 player_y: DS 1
 enemy_x:  DS 1
 enemy_y:  DS 1
SECTION "Code", ROM0
;;-------------------------------------------------
;; Checks collisions between player and enemy
;;   Colision if (px, py) == (ex, ey)
;;   (x,y) are tile coordinates
;; ↩️ Return value --
;;   A  = 0, when NOT colliding
;;   A != 0, when colliding
check_collision:
  ;; Your code here
  ret
```